CS344 Assignment 2 Group 6

Adarsh Gupta	220101003	Aditya Nanda	220101006
Aryan Raj	220101019	Ashish Raj	220101020

Drive Link of Patch Files:
https://drive.google.com/drive/folders/1lu1TKlBgFjNcx9WGqebTyUeawt71v_An?usp=drive_link

Part A.

1. Add settickets system call

For adding the settickets system call, a new entry needed to be added to the proc struct: Namely tickets.

```
// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
```

For adding the system call, the following changes were made:

1. syscall.h

```
#define SYS_settickets 22
```

2. syscall.c

```
extern int sys_settickets(void);
```

```
[SYS_settickets] sys_settickets,
```

3. sysproc.c & proc.c (NOTE: setticket prototype also added to defs.h)

```
int
sys_settickets(void)
{
  int numtickets;

  // Getting integer argument: number of tickets
  int resp0 = argint(0, &numtickets);
  if(resp0 < 0) return -1;

  return setticket(numtickets);
}
```

```
int setticket(int numtickets){
  acquire(&ptable.lock);
  struct proc* p = myproc();
  p->tickets = numtickets;
  release(&ptable.lock);
  return numtickets;
}
```

4. user.h

```
int settickets(int);
```

5. usys.S

```
SYSCALL(settickets)
```

Testing is done using testwithtickets below (See 2. Testing. 2)

2. Add getprocessinfo system call

IMPLEMENTATION:

The following modifications were made:

1. ticks field added to proc struct
2. Added processes_info struct (as defined in assignment) to proc.h

```
// Required for lottery scheduling
struct processes_info{
  int num_processes;           // Number of non-UNUSED processes
  int pids[NPROC];             // pids of  non-UNUSED processes
  int ticks[NPROC];            // number of ticks a process is scheduled
  int tickets[NPROC];          // number of tickets set by settickets()
};
```

3. For adding the system call: basic modification to syscall.h, syscall.c, user.h and usys.S. Actual code added in sysproc.c & proc.c:

```
int
sys_getprocessesinfo(void)
{
  void *processInfoPtr;

  int resp0 = argptr(0, (void*)&processInfoPtr, sizeof(processInfoPtr));
  if(resp0 < 0) return -1;

  int resp = populate_processes_info((struct processes_info*) processInfoPtr);
  return resp;
}
```

```c
int populate_processes_info(struct processes_info* processes_info){
  acquire(&ptable.lock);
  struct proc* p;
  int proc_count = 0;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state == UNUSED)
      continue;

    processes_info->pids[proc_count] = p->pid;
    processes_info->ticks[proc_count] = p->ticks;
    processes_info->tickets[proc_count] = p->tickets;
    proc_count++;
  }
  processes_info->num_processes = proc_count;
  release(&ptable.lock);
  return 1;
}
```

NOTE: populate_processes_info, processes_info struct prototype added to defs.h for further linking by the compiler. processes_info struct prototype also added to user.h to allow users to create processes_info pointer for using the system call.

TESTING: (Tested after implementing lottery scheduler)

1. For testing getprocessesinfo, a test program has been provided, which outputs the processes_info struct.

```c
#include "types.h"
#include "mmu.h"
#include "param.h"
#include "proc.h"
#include "user.h"
int main(int argc, char *argv[])
{
    struct processes_info info;
    getprocessesinfo(&info);
    printf(1, "%d running processes\n", info.num_processes);
    printf(1, "PID\tTICKETS\tTICKS\n");
    for (int i = 0; i < info.num_processes; ++i) {
        printf(1, "%d\t%d\t%d\n", info.pids[i], info.tickets[i], info.ticks[i]);
    }
    exit();
}
```

```
$ processlist
3 running processes
PID     TICKETS TICKS
1       10      24
2       10      21
4       10      8
```

2. The second testing file provided is timewithtickets.c (NOTE: timewithtickets program also required a yield system call, which isn't natively added in xv6, I just made a wrapper around the already existing yield call)

```
$ timewithtickets
usage: timewithtickets seconds tickets1 tickets2 ... ticketsN
       seconds is the number of time units to run for
       ticketsX is the number of tickets to give to subprocess N
$ timewithtickets 1000 1000 100 1
TICKETS TICKS
1000    921
100     78
1       1
```

The approximate ticks ratio observed is similar to what we expected.

3. Adding Lottery Scheduler

PseudoRandom Number Generator:
Used the provided Park-Miller pseudo-random number generator, and added a utility function over it, which regenerates the numbers to avoid bias due to overlap.

```c
// Random number generation for lottery scheduling
static unsigned random_seed = 1;

#define RANDOM_MAX ((1u << 31u) - 1u)
unsigned lcg_parkmiller(unsigned *state)
{
    const unsigned N = 0x7fffffff;
    const unsigned G = 48271u;

    unsigned div = *state / (N / G);  /* max : 2,147,483,646 / 44,488 = 48,271 */
    unsigned rem = *state % (N / G);  /* max : 2,147,483,646 % 44,488 = 44,487 */

    unsigned a = rem * G;        /* max : 44,487 * 48,271 = 2,147,431,977 */
    unsigned b = div * (N % G);  /* max : 48,271 * 3,399 = 164,073,129 */

    return *state = (a > b) ? (a - b) : (a + (N - b));
}

unsigned next_random() {
    return lcg_parkmiller(&random_seed);
}

/* Generates an unbiased random number in the range [0, numtickets) */
unsigned random_in_range(unsigned numtickets) {
    unsigned x;
    unsigned limit = RANDOM_MAX - (RANDOM_MAX % numtickets);

    // Rejection sampling to remove bias
    do {
        x = next_random();
    } while (x >= limit);

    return x % numtickets;
}
```

Scheduler Code: The logic used is to count the total number of ticket by iterating through all the processes, then generating a winning ticket number by using the above pseudo random number generator, then we iterate through the processes and decrement this winner ticket until it gets below 0 (this logic is analogous to maintaining a counter variable and incrementing it and checking when it gets above winning ticket)

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  int totalTickets = 0;
  int winningTicket, currentTicket;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);

    // Find the total number of tickets of RUNNABLE processes
    totalTickets = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
      if(p->state == RUNNABLE)
        totalTickets += p->tickets;
    if(totalTickets == 0){
      release(&ptable.lock);
      continue;
    }

    // Generate a random number between [0, totaltickets)
    winningTicket = random_in_range(totalTickets);
```

```
    // Select winner using the winningTicket number
    currentTicket = winningTicket;
    struct proc* winner = ptable.proc;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;
      currentTicket -= p->tickets;
      if(currentTicket >= 0)
        continue;
      else{
        winner = p;
        // Switch to winner process
        c->proc = winner;
        switchuvm(winner);
        winner->state = RUNNING;
        winner->ticks++;

        swtch(&(c->scheduler), winner->context);
        switchkvm();

        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;

        break;
      }
    }

    release(&ptable.lock);
  }
}
```

Testing:

A lotterytest program has been provided (NOTE: To get it to work, some changes needed to be made to the makefile like linking the gcc-multifile object)

```
$ lotterytest
one process: passed 3 of 3
two processes, unequal ratio: passed 7 of 7
two processes, unequal ratio, small ticket count: passed 7 of 7
two processes, equal: passed 7 of 7
two processes, equal, small ticket count: passed 7 of 7
three processes, unequal: passed 9 of 9
three processes, unequal, small ticket count: passed 9 of 9
three processes, but one io-wait: passed 9 of 9
three processes, but one exits: passed 9 of 9
seven procceses: passed 17 of 17
two processes, not all yielding: passed 7 of 7
overall: passed 91 of 91
```

Part B.

1. System Call for Retrieving Active Process Information

In this part of the project, we introduced two new system calls: `getNumProc()` and `getMaxPid()` with the help of `getNumProc Assist()` and `getMaxPidAssist()`. These system calls allow us to obtain information about the currently active processes in the xv6 system, such as the total number of processes and the maximum PID of all active processes.

getNumProc():

- This system call returns the total number of active processes in the system calling getNumProcAssist(). Active processes include those in states like embryo, runnable, running, sleeping, and zombie.
- We iterate over the process table `ptable` and count processes that are not in the `UNUSED` state.
- Before reading or writing to the process table, the `ptable.lock` is acquired to ensure thread safety.

getMaxPid():

- This system call calls the function getMaxPIDAssist() returns the maximum PID amongst all active processes in the system.
- Similar to `getNumProcAssist()`, the process table is iterated, but in this case, the highest PID value is identified.

```
int
sys_getNumProc(void)
{

  return getNumProcAssist();

}

int
sys_getMaxPID(void)
{

  return getMaxPIDAssist();

}
```

The above code in sysproc.c is calling the helping function in proc.c which implement the logic.

```
int getNumProcAssist(void){

  int ans=0;
  struct proc *p;

  acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != UNUSED)
        ans++;
    }
    release(&ptable.lock);

    return ans;
}

int getMaxPIDAssist(void){

  int max=0;
  struct proc *p;

  acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != UNUSED){
        if(p->pid > max)
          max=p->pid;
      }
    }
    release(&ptable.lock);

    return max;
}
```

Two user programs were created to test the system calls `getNumProc()` and `getMaxPid()`.

The getMaxPIDTest user program is run.

- The next available PID is 4, which is assigned to getMaxPIDTest.
- At this point, there are 3 active processes in the system:
  - System process with PID 1.
  - System process with PID 2.
  - getMaxPIDTest with PID 4.
- Since the highest PID in the system is 4, the output of getMaxPIDTest is **4**.

the `getNumProcTest` program is executed.

- Similar to the previous scenario, `getNumProcTest` also has PID 4, which remains the next available PID after `ls` terminates.
- Hence, the output of `getNumProcTest` is **3**

```
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00



Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ ls
.              1 1 512
..             1 1 512
README         2 2 2286
cat            2 3 15544
echo           2 4 14420
forktest       2 5 8864
grep           2 6 18388
init           2 7 15044
kill           2 8 14508
ln             2 9 14404
ls             2 10 16976
mkdir          2 11 14528
rm             2 12 14508
sh             2 13 28568
stressfs       2 14 15440
usertests      2 15 62944
wc             2 16 15964
zombie         2 17 14088
getNumproc_tes 2 18 14196
getMaxPid_test 2 19 14192
console        3 20 0
$ getMaxPid_test
The maximum PID of all active processes in the system is 4
$ getNumproc_test
The total number of active processes in the system is 3
```

2. System Call for Retrieving Process Information by PID

The second part of the project involves implementing the `getProcInfo(pid, &processInfo)` system call. This call returns detailed information about a specific process, identified by its PID, including the parent process ID, the process size, and the number of times the process has been context-switched by the scheduler.

- The `processInfo` structure, defined in the provided `processInfo.h` header, contains fields for the parent PID (`ppid`), the process size in bytes (`psize`), and the number of context switches (`numberContextSwitches`).
- To gather this information, we iterate through the process table and match the given PID. If a match is found, the relevant information is filled into the `processInfo` structure.
- If no process with the given PID exists, the system call returns an error code `-1`.

`processInfo.h`

```
#define PROCESS_INFO_H

struct processInfo
{
    int ppid;
    int psize;
    int numberContextSwitches;
};

#endif
```

System call getProcInfo() calls getProcInfoAssist() present in proc.c which implements the above logic.

sysproc.c

```
int
sys_getProcInfo(void){
  int pid;

  struct processInfo *info;
  argptr(0,(void *)&pid, sizeof(pid));
  argptr(1,(void *)&info, sizeof(info));

  struct processInfo temporaryInfo = getProcInfoAssist(pid);

  if(temporaryInfo.ppid == -1)return -1;

  info->ppid = temporaryInfo.ppid;
  info->psize = temporaryInfo.psize;
  info->numberContextSwitches = temporaryInfo.numberContextSwitches;
  return 0;

}
```

proc.c

```
struct processInfo getProcInfoAssist(int pid){

  struct proc *p;
  struct processInfo temp = {-1,0,0};

  acquire(&ptable.lock);
    for(p = pt                    &ptable.proc[NPROC]; p++){
      if(p->st
        // pri                    p->pid);
        if(p->pid == pid) {
          temp.ppid = p->parent->pid;
          temp.psize = p->sz;
          temp.numberContextSwitches = p->nocs;
          release(&ptable.lock);
          return temp;
        }
      }
    }
    release(&ptable.lock);

    return temp;

}
```

int proc::pid

Process ID

To store the number of context switches for a process, a new field named `nocs` (Number of Context Switches) is added to the `struct proc` in xv6. The following changes were made to implement this feature:

- **Initializing `nocs`:**
  The `nocs` field is initialized to 0 in the `allocproc()` function, which allocates a new process in the process table (`ptable`).
- **Updating `nocs`:**
  In the `scheduler()` function, the `nocs` value is incremented each time the process is scheduled, i.e., every time it undergoes a context switch. This ensures that the field is continuously updated as the process runs.

To manage parent-child relationships, a dummy process called `defaultParent` is created. It serves as a placeholder for processes that do not yet have a parent assigned during their allocation phase. The following steps were taken:

- **Assigning Default Parent:**
  In the `allocproc()` function, every new process is initially assigned `defaultParent` as its parent using `p->parent = &defaultParent`.
- **Replacing with Actual Parent:**
  When a process is forked, its actual parent is assigned using the `fork()` function, replacing `defaultParent`.
- **Handling Orphan Processes:**
  If a process has no valid parent, `p->parent->pid` will return `-2`, as the PID of `defaultParent` is set to this value.

getProcInfoTest.c

```c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char* argv[]){

    if (argc < 2)
    {
        printf(1, "Incorrect Format!\nFormat: getProcInfoTest <PID>\n");
        exit();
    }

    struct processInfo info;
    if(getProcInfo(atoi(argv[1 (char [26])"No process has that PID.\n"
        printf(1, "No process has that PID.\n");
        exit();
    }
    if(info.ppid != -2)
        printf(1, "PPID: %d\nSize: %d\nNumber of Context Switches: %d\n", info.ppid, info.psize, info.numberContextSwitches);
    else
        printf(1, "PPID: No Parent Process\nSize: %d\nNumber of Context Switches: %d\n", info.psize, info.numberContextSwitches);
    exit();
}
```

```
SeaBIOS (version 1.15.0-1)


iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00



Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ getProcInfoTest 1
PPID: No Parent Process
Size: 16384
Number of Context Switches: 24
$ getProcInfoTest 2
PPID: 1
Size: 20480
Number of Context Switches: 20
$ getProcInfoTest 3
No process has that PID.
$
```

3. System Calls for Setting and Getting Process Burst Time

In this part of the project, we modify the process structure to accommodate a burst time field and introduce system calls to set and retrieve the burst time for each process. These system calls will be used to define an approximate time that each process should run for. To store the burst time, a new attribute, `burst_time`, was added to the `struct proc`. This is done in the `proc.h` file. Each process is allocated using the `allocproc()` function. Before any process starts executing, we initialize its burst time to a default value of 0.

set_burst_time(n):

This system call sets the burst time for the calling process to the given integer `n`. The burst time indicates how much CPU time the process should get (although in this part, we simply store the value without using it for scheduling).

get_burst_time():

This system call returns the burst time previously set for the calling process, allowing users to verify that the burst time was stored correctly.

```
int
sys_set_burst_time(void)
{
  int burst_time;
  argptr(0,(void *)&burst_time, sizeof(burst_time));

  return set_burst_timeAssist(burst_time);
}

int
sys_get_burst_time(void)
{
  return get_burst_timeAssist();
}
```

proc.h

```
struct proc {
  uint sz;                     // Size of process memory (byte
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for t
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current sysca
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on cha
  int killed;                  // If non-zero, have been kille
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  int nocs;                    //Number of context switches
  int burst_time;              //Burst time duhhh
  int rt;                      //running time

};
```

proc.c

```
int
set_burst_timeAssist(int burst_time)
{
  struct proc *p = myproc();
  p->burst_time = burst_time;
  yield();

  return 0;
}

int
get_burst_timeAssist()
{
  struct proc *p = myproc();

  return p->burst_time;

}
```

To test a simple program burstTimeTest.c

```c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void){
    printf(1,"This is a sample process to test set_burst_time and get_burst_time system calls.\n");
    set_burst_time(3);
    printf(1, "The burst time is: %d\n", get_burst_time());
    exit();
}
```

Output:

```
SeaBIOS (version 1.15.0-1)


iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00



Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ burstTimeTest
This is a sample process to test set_burst_time and get_burst_time system calls.
The burst time is: 3
$
```

Part C.

Remove Preemption (Disabling Clock Tick Yield):

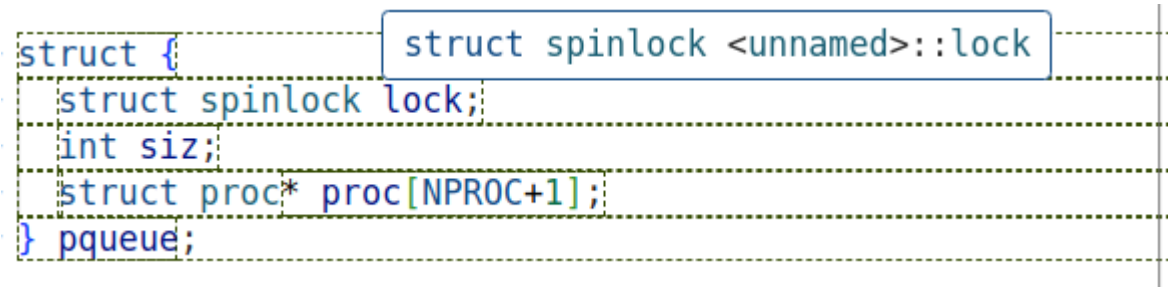
In a Round Robin scheduler, the process gets preempted at regular clock ticks. For SJF, we need to allow the current process to fully complete its execution before switching to the next. This means removing the forced preemption mechanism from `trap.c`, which occurs on every OS clock tick.

```c
  if(myproc() && myproc()->state == RUNNING &&
     tf->trapno == T_IRQ0+IRQ_TIMER)
    yield();
```

Modifications for Scheduler:

1. **Data Structure for Process Management:**
   - I've modified the `ptable` (process table) to include a priority queue (`pqueue`) using a min-heap for efficient retrieval of the process with the smallest burst time.
   - The priority queue supports operations like insertion (`insertIntoHeap()`), extraction of the minimum element (`extractMin()`), and burst time update (`changeKey()`).
2. **Scheduler Logic (SJF Algorithm):**
   - The SJF scheduler picks the process with the shortest burst time to execute next. This is implemented by modifying the `scheduler()` function to use `extractMin()` to fetch the next process to run.
   - Processes are inserted into the priority queue upon becoming `RUNNABLE`, using `insertIntoHeap()`.
   - Burst times can be adjusted dynamically with `changeKey()`.
3. **Concurrency and Locking:**
   - Locks (`spinlock`) are used to ensure that updates to shared structures like `ptable` and `pqueue` are thread-safe, especially when running over multiple CPUs.

Declare priority queue:



Insert into Priority Queue: When a process becomes `RUNNABLE`, it should be inserted into the priority queue based on its burst time. The insertion function should maintain the heap properties to ensure that the queue is sorted by burst time.

```
void insertIntoHeap(struct proc *p){
  if(isFull())
    return;

  acquire(&pqueue.lock);

  pqueue.siz++;
  pqueue.proc[pqueue.siz]=p;
  int curr=pqueue.siz;
  while(curr>1 && ((pqueue.proc[curr]->burst_time)<(pqueue.proc[curr/2]->burst_time))){
    struct proc* temp=pqueue.proc[curr];
    pqueue.proc[curr]=pqueue.proc[curr/2];
    pqueue.proc[curr/2]=temp;
    curr/=2;
  }
  release(&pqueue.lock);


}
```

struct <unnamed> pqueue

IsFull: Checks if the priority queue is empty or not.

```
int isFull(){
  acquire(&pqueue.lock);
  if(pqueue.siz==NPROC){
    release(&pqueue.lock);
    return 1;
  }
  else{
    release(&pqueue.lock);
    return 0;
  }
}
```

IsEmpty: Checks if the priority queue is empty or not.

```
int isEmpty(){
  acquire(&pqueue.lock);
  if(pqueue.siz == 0){
    release(&pqueue.lock);
    return 1;
  }
  else{
    release(&pqueue.lock);
    return 0;
  }
}
```

extractMin: removes the process at the front of the queue and returns it

```
struct proc * extractMin(){

  if(isEmpty())
    return 0;

  acquire(&pqueue.lock);
  struct proc* min=pqueue.proc[1];
  if(pqueue.siz==1)
  {
    pqueue.siz=0;
    release(&pqueue.lock);
  }
  else{
    pqueue.proc[1] = pqueue.proc[pqueue.siz];
    pqueue.siz--;
    release(&pqueue.lock);

    fix(1);
  }
  return min;
}
```

changeKey: Changes the burst time of a process with a given PID in the priority queue and updates the queue accordingly):

```
void changeKey(int pid, int x){

  acquire(&pqueue.lock);

  struct proc* p;
  int curr=-1;
  for(int i=1;i<=pqueue.siz;i++){
    if(pqueue.proc[i]->pid == pid){
      p=pqueue.proc[i];
      curr=i;
      break;
    }
  }

  if(curr==-1){
    release(&pqueue.lock);
    return;
  }


  if(curr==pqueue.siz){
    pqueue.siz--;
    release(&pqueue.lock);
  }
  else{
    pqueue.proc[curr]=pqueue.proc[pqueue.siz];
    pqueue.siz--;
    release(&pqueue.lock);

    fix(curr);
  }

  p->burst_time=x;
  insertIntoHeap(p);

}
```

Fix: performs Heapify on priority queue - basically converts the array into min heap assuming that the left subtree and the right subtree of the root are already min heaps.

```
void fix(int curr){

  acquire(&pqueue.lock);
  while(curr*2<=pqueue.siz){
    if(curr*2+1<=pqueue.siz){
      if((pqueue.proc[curr]->burst_time)<=(pqueue.proc[curr*2]->burst_time)&&
          ((pqueue.proc[curr]->burst_time)<=(pqueue.proc[curr*2+1]->burst_time))
        break;
      else{
        if((pqueue.proc[curr*2]->burst_time)<=(pqueue.proc[curr*2+1]->burst_time)){
          struct proc* temp=pqueue.proc[curr*2];
          pqueue.proc[curr*2]=pqueue.proc[curr];
          pqueue.proc[curr]=temp;
          curr*=2;
        } else {
          struct proc* temp=pqueue.proc[curr*2+1];
          pqueue.proc[curr*2+1]=pqueue.proc[curr];
          pqueue.proc[curr]=temp;
          curr*=2;
          curr++;
        }
      }
    } else {
      if((pqueue.proc[curr]->burst_time)<=(pqueue.proc[curr*2]->burst_time))
        break;
      else{
        struct proc* temp=pqueue.proc[curr*2];
        pqueue.proc[curr*2]=pqueue.proc[curr];
        pqueue.proc[curr]=temp;
        curr*=2;
      }
    }
  }
  release(&pqueue.lock);

}
```

struct proc *<unnamed>::proc[65]

Scheduler Logic (SJF Implementation):

Modify the `scheduler()` function in `proc.c` to pick the next process using the `extractMin()` function. Priority queue locks are acquired and released.

```
void
scheduler(void)
{

  defaultParent.pid = -2;
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();


    acquire(&ptable.lock);

    //NEW SJF SCHEDULER

    if((p = extractMin()) == 0){release(&ptable.lock);continue;}

    if(p->state!=RUNNABLE)
      {release(&ptable.lock);continue;}



    c->proc = p;
    switchuvm(p);

    p->state = RUNNING;
    (p->nocs)++;

    swtch(&(c->scheduler), p->context);

    switchkvm();


    c->proc = 0;

    release(&ptable.lock);

  }
}
```

To integrate the SJF scheduler and handle process insertion into the priority queue effectively, we need to make sure that the processes are inserted into the priority queue when their state becomes RUNNABLE. This typically happens in functions like `wakeup1`, `exit`, `kill`, and `fork`, as well as in the function responsible for setting the burst time (`set_burst_time`).
The code from the fork function is given below. The rest of the instances are identical. The variable check is created to check if the process was already in the RUNNABLE state in which case it is already in the priority queue and shouldn't be inserted again.

```
  acquire(&ptable.lock);

  short check = (np->state!=RUNNABLE);
  np->state = RUNNABLE;

  //Insert Process Into Queue
  if(check)
    insertIntoHeap(np);

  release(&ptable.lock);
```

Insert a yield call into set_burst_timeAssist. This is because when burst time of a process is set, its scheduling needs to be done on the basis of the new burst time. yield switches the state of the current process to RUNNABLE, inserts it into the priority queue and switches the context to the scheduler.

```
int
set_burst_timeAssist(int burst_time)
{
  struct proc *p = myproc();
  p->burst_time = burst_time;
  yield();

  return 0;
}
```

Runtime Complexity

- The overall time complexity of the scheduler remains `O(log n)`, where `n` is the number of `RUNNABLE` processes, due to the `extractMin` and

`insert_plq` operations. These heap-based operations (insertion and extraction) each take `O(log n)` time.
- Other operations within the scheduler (e.g., checking process states) are `O(1)`.

Corner Case Handling and Safety

The following safeguards ensure system stability:

- If the priority queue is empty (i.e., no `RUNNABLE` processes), `extractMin` will return `0`, and no scheduling will occur.
- If the priority queue is full, `insertIntoHeap` rejects new processes without causing an overflow.
- Before inserting a process, we always check if it's already present in the queue to prevent duplicates.
- The scheduler verifies if the process at the front of the queue is still `RUNNABLE` before scheduling it. This ensures that if a process changes its state between selection and execution, the scheduler won't attempt to run it.
- ZOMBIE processes are removed from the queue upon being freed, preventing them from lingering in the priority queue.
- A dedicated lock (`ptable.lock` or a custom queue lock) ensures thread safety when accessing the priority queue.
- This lock is created specially for pTXeXe and is initialised in pinit:

```
void
pinit(void)
{
  initlock(&ptable.lock, "ptable");
  initlock(&pqueue.lock, "pqueue");
}
```

Testing:

The goal of this test was to evaluate and compare the performance of the Round Robin (RR) and Shortest Job First (SJF) scheduling algorithms on CPU-bound and I/O-bound processes. The performance metrics focused on burst times and context switches, with an emphasis on the fairness of execution in Round Robin versus the efficiency in SJF.

Test Setup

Two sets of test programs were implemented to generate CPU-bound and I/O-bound processes:

1. `testscheduler.c`: Mix of CPU-bound and I/O-bound processes.
2. `cpuproctest.c`: Exclusively CPU-bound processes.
3. `ioproctest.c`: Exclusively I/O-bound processes.

Each test involved:

- Forking multiple child processes, each having a random burst time between 1 and 1000 ms.
- CPU-bound processes performed intensive computation.
- I/O-bound processes used `sleep()` to simulate I/O wait time.
- Each process tracked and reported the number of context switches.

All CPU bound processes and I/O bound processes are sorted by their burst times and CPU bound processes finish first. The CPU bound processes finish first because I/O bound processes are blocked by the ‘sleep’ system call. Since the processes are sorted by burst time, we can say that the SJF scheduler is working perfectly. The context switches are also as expected. In the case of CPU bound processes, first the process is switched in after which **set_burst_time** is called because of which the process is yielded and the next process is brought in. Finally, when the earlier process is chosen again by the scheduler, it finishes. In the case of I/O bound processes, they are also put to sleep 100 times. Hence, the processes have 100 additional context switches (they are brought back in 100 more times).

```
Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ test_scheduler 20
       PID      Type    Burst Time      Context Switches
       ___      ____    __________      ________________

       21       CPU         2                2
       11       CPU         3                2
       7        CPU         6                2
       13       CPU         6                2
       17       CPU         9                2
       19       CPU         14               2
       15       CPU         14               2
       23       CPU         17               2
       9        CPU         20               2
       5        CPU         20               2
       6        I/O         1                102
       8        I/O         2                102
       10       I/O         6                102
       20       I/O         9                102
       4        I/O         13               102
       22       I/O         15               102
       12       I/O         17               102
       18       I/O         18               102
       16       I/O         19               102
       14       I/O         20               102
```

SJF Scheduler:

```
cpuProcTester 4
      PID      Type    Burst Time      Context Switches
      ___      ____    __________      ________________

      27       CPU         1                2
      28       CPU         6                2
      25       CPU         13               2
      26       CPU         20               2
```

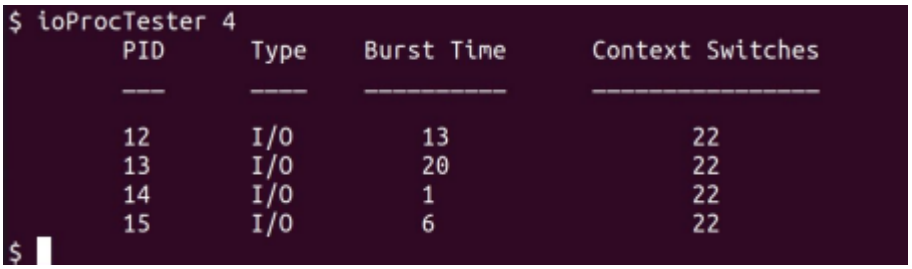
```
ioProcTester 4
      PID      Type    Burst Time      Context Switches
      ___      ____    __________      ________________

      32       I/O         1                22
      33       I/O         6                22
      30       I/O         13               22
      31       I/O         20               22
```

Round Robin Scheduler:

```
$ cpuProcTester 4
       PID      Type    Burst Time      Context Switches
       ___      ____    __________      ________________

       4        CPU         13               17
       5        CPU         20               18
       6        CPU         1                18
       7        CPU         6                19
$
```



Since the Round Robin scheduler uses an FCFS queue, the order of execution is highly related to the PID of the process whereas in the SJF scheduler, the scheduling is happening by the burst times. Also, the number of context switches in the RR scheduler is very high. This is because of forced preemption on every clock tick.

Burst times are generated by the random number generator created in the file random.c. We made his file a user library

Part C (Bonus):

We designed a hybrid scheduler combining aspects of Round Robin (RR) and Shortest Job First (SJF) by making modifications to `trap.c`, `proc.c`, and `defs.h`. The main focus was adjusting the time quantum based on the process with the shortest burst time.

Step 1: Setting Up the Time Quantum

To establish a time quantum, we introduced an external integer variable called `quant` in `proc.c`. This variable starts with a default value of 1000 (as burst times range between 1 and 1000). The assignment required us to set the time quantum equal to the burst time of the shortest job in the priority queue. If a process's burst time is zero, which makes it undefined, we apply a default burst time of zero. Hence, the `quant` variable is updated in the `set_burst_time` function. If a process's burst time is less than `quant`, the value of `quant` is set to the process's burst time.

Step 2: Creating a New Priority Queue

We implemented a new priority queue called `pqueue2` with functions like `insertIntoHeap2`, `extractMax2`, and `fix2`, which mimic the behavior of `pqueue`. These new functions are used in place of `pqueue` for managing processes with priority-based scheduling, allowing for flexible handling of tasks.

Step 3: Modifying the Clock Tick Handler

We adjusted the clock tick interrupt handler within `trap.c` to track the running time of processes. A new field `rt` (running time) was added to the `proc` structure, initialized to zero when the process is created. This tracks the running time and is incremented on each clock tick.

For processes with a manually assigned burst time, we check whether their running time equals their burst time. If a match occurs, the process is terminated (`exit()`). Otherwise, if the process's runtime is divisible by the time quantum (`quant`), the process is preempted and re-inserted into `pqueue2`. This ensures processes are fairly scheduled according to their burst times.

The system avoids terminating processes on their first clock tick to prevent system instability, allowing for balanced scheduling decisions between preemption (Round Robin style) and execution based on burst time (SJF style).

```
// Force process to give up CPU on clock tick.
// If interrupts were on while locks held, would need to check nlock.
if(myproc() && myproc()->state == RUNNING &&
   tf->trapno == T_IRQ0+IRQ_TIMER)
{

  (myproc()->rt)++;
  if(myproc()->burst_time != 0){

    if(myproc()->burst_time == myproc()->rt)
      exit();
  }
  if((myproc()->rt)%quant == 0)
      new_yield();


}
```

```
void new_yield(void){

  acquire(&ptable.lock);

  myproc()->state = RUNNABLE;

  insertIntoHeap2(myproc());

  sched();
  release(&ptable.lock);

}
```

We updated the scheduler to handle processes based on the priority queues. When deciding which process to run next, the new scheduler first checks if the original priority queue, `pqueue`, is empty. If it is not empty, the process with the highest priority is extracted using `extractMin` from `pqueue`. If `pqueue` is empty, all processes are moved from `pqueue2` (containing preempted processes) into `pqueue`, and the first element is extracted using `extractMin`.

```
void
scheduler(void)
{

  defaultParent.pid = -2;
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

    if(isEmpty()){
      if(isEmpty2()){
        goto label;
      }


      while(!isEmpty2()){
        if((p = extractMin2()) == 0){release(&ptable.lock);break;}
        insertIntoHeap(p);
      }

    }

    label:

    if((p = extractMin()) == 0){release(&ptable.lock);continue;}

    if(p->state!=RUNNABLE)
      {release(&ptable.lock);continue;}

    c->proc = p;
    switchuvm(p);

    p->state = RUNNING;
    (p->nocs)++;

    swtch(&(c->scheduler), p->context);
```

Testing

We tested the scheduler using three user programs: `test_scheduler`, `cpuProcTester`, and `ioProcTester`. These programs were used to simulate different types of processes and validate the scheduler's functionality under different conditions:

- **test_scheduler**: This test forks processes where half are I/O-bound and the other half are CPU-bound. Each process is assigned a random burst time between 1 and 1000. CPU-bound processes perform loops running 10910^9109 iterations, while I/O-bound processes simulate I/O by calling `sleep(1)` ten times.

```
$ test_scheduler 30
        PID      Type    Burst Time      Context Switches
        ---      ----    ----------      ----------------

        48       I/O         21                12
        50       I/O         71                12
        72       I/O         145                12
        66       I/O         179                12
        52       I/O         252                12
        62       I/O         411                12
        74       I/O         423                12
        46       I/O         635                12
        64       I/O         738                12
        68       I/O         805                12
        54       I/O         822                12
        60       I/O         889                12
        70       I/O         914                12
        58       I/O         932                12
        56       I/O         962                12
        55       CPU         270                9
        69       CPU         418                9
        73       CPU         457         75     CPU         571         49        CPU         0
        59       CPU         443                10
          9
            9
        67       CPU         624                9
        57       CPU         675                10
        61       CPU         677                9
        65       CPU         846                10
        51       CPU         985                9
        47       CPU         999                10
```

- **cpuProcTester**: In this test, all forked processes are CPU-bound with burst times assigned randomly between 1 and 1000. Each process runs loops of 10910^9109 iterations, which takes approximately 200 ticks.
- **ioProcTester**: In this case, every forked process is I/O-bound. I/O is simulated using `sleep(1)` ten times, and the burst times are randomly assigned between 1 and 1000.

The report presents the results of a hybrid scheduling experiment with several observations:

1. **Not all CPU-bound processes were completed**: Some had a burst time lower than their actual execution time and were terminated before they could finish, which suggests that when the remaining time of a process equals its burst time, it is exited.
2. **Increased context switches in CPU-bound processes**: This occurred because CPU-bound processes were frequently preempted at every clock tick (quantum), leading to more context switching compared to standard Shortest Job First (SJF) scheduling.
3. **I/O-bound processes were unaffected by preemption**: They behaved as they would in SJF scheduling since they spent most of their time sleeping. Their actual execution time was much lower, with minimal preemption due to the low likelihood of encountering quant clock ticks.

```
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ cpuProcTester 20
        PID      Type    Burst Time      Context Switches
        ___      ____    __________      ________________

        10       CPU         252                 9
        7        CPU         264                 10
        13       CPU         270                 10
        20       CPU         411                 10
        17       CPU         443                 10
        4        CPU         635                 10
        15       CPU         675                 10
        19       CPU         677                 10
        22       CPU         738                 10
        12       CPU         822                 10
        23       CPU         846                 10
        18       CPU         889                 10
        16       CPU         932                 10
        14       CPU         962                 10
        9        CPU         985                 10
        5        CPU         999                 10
```

```
$ ioProcTester 20
        PID      Type    Burst Time      Context Switches
        ___      ____    __________      ________________

        27       I/O         21                  12
        42       I/O         67                  12
        29       I/O         71                  12
        32       I/O         126                  12
        31       I/O         252                  12
        28       I/O         264                  12
        34       I/O         270                  12
        41       I/O         411                  12
        38       I/O         443                  12
        25       I/O         635                  12
        36       I/O         675                  12
        40       I/O         677                  12
        43       I/O         738                  12
        33       I/O         822                  12
        44       I/O         846                  12
        39       I/O         889                  12
        37       I/O         932                  12
        35       I/O         962                  12
        30       I/O         985                  12
        26       I/O         999                  12
```